

Web Development Practical Cheat Sheet

Double-sided revision guide: Fetch • Loops • DOM • Layout • Classes • Events • Forms

1. The core workflow

Memorise: Select → Fetch → JSON → Loop → Build string → **innerHTML once** → Events.

```
const grid = document.querySelector("#card-grid");

async function getData() {
  const response = await fetch("API_URL");
  const data = await response.json();
  render(data);
}

getData();
```

Fetch: asks for data. **await:** waits for it. **.json():** converts JSON into usable JavaScript. An **async** function is required when you use **await**.

2. Loop, build, and display API data

```
function render(data) {
  let html = "";

  data.forEach(item => {
    html += `
    <article class="card">
    
    <h2>${item.title}</h2>
    <p>${item.description}</p>
    </article>`;
  });

  grid.innerHTML = html;
}
```

Key rule: Use backticks for template literals and put variables inside **\${...}**. Use **+=** in the loop, then assign **innerHTML** once after the loop.

Dot notation: `item.title`, `item.author`, `item.cover_url`. For one object: `sampleData.title`.

3. Selecting DOM elements

```
const title = document.querySelector("#title");
const button = document.querySelector(".button");
const cards = document.querySelectorAll(".card");
```

means ID; . means class. **querySelector** gets the first matching item. **querySelectorAll** gets all matches, so normally use **forEach**.

Changing content: `element.textContent = "New text";`

Adding HTML: `element.innerHTML = html;`

4. Flexbox and Grid

```
/* Side-by-side featured item */
#featured {
  display: flex;
  gap: 20px;
  align-items: flex-start;
}
#featured img { width: 180px; height: auto; flex-shrink: 0; }

/* Three-column card grid */
#card-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
.card {
  display: flex;
  flex-direction: column;
  padding: 20px;
  border-radius: 8px;
  background: white;
}
.card img { width: 140px; height: auto; }
```

Flexbox = one-dimensional layout (row or column). **Grid** = rows and columns. **gap** creates space between items. **1fr** means one equal fraction of available width.

Web Development Practical Cheat Sheet

Side 2 — interactions, forms, troubleshooting, and exam checklist

5. CSS classes: show/hide content

```
/* CSS */
.description { display: none; }
.description.visible { display: block; }

/* JavaScript */
const description = document.querySelector(".description");
description.classList.toggle("visible");
```

classList methods: add("visible"), remove("visible"), toggle("visible"), contains("visible").

6. Event listeners

```
const button = document.querySelector("#toggle-desc");
const description = document.querySelector("#featured .description");

button.addEventListener("click", () => {
  description.classList.toggle("visible");
  button.textContent = description.classList.contains("visible")
    ? "Show less"
    : "Show more";
});
```

Format: element.addEventListener("event", () => { ... });

Common events: "click" for a button, "change" for checkbox/select, "input" while typing, "submit" for a form.

7. Checkbox: change all cards

```
const checkbox = document.querySelector("#show-descriptions");

checkbox.addEventListener("change", () => {
  const descriptions = document.querySelectorAll(
    ".card .description"
  );

  descriptions.forEach(description => {
    description.classList.toggle("visible", checkbox.checked);
  });
});
```

Important: select card descriptions inside the event listener if cards are made by an API fetch. They do not exist until after the fetch finishes.

```
<input type="checkbox" id="show-descriptions">
<label for="show-descriptions">Show descriptions</label>
```

8. Forms and user input

```
const form = document.querySelector("#search-form");
const input = document.querySelector("#search-input");

form.addEventListener("submit", event => {
  event.preventDefault();
  const searchTerm = input.value.trim().toLowerCase();

  const filtered = allData.filter(item =>
    item.title.toLowerCase().includes(searchTerm)
  );

  render(filtered);
});
```

input.value gets what the user typed. **preventDefault()** stops the form refreshing the page. **trim()** removes accidental spaces. **filter()** makes a new array containing only matches.

9. Quick fixes and syntax reminders

- CSS assignment uses `=`: `document.body.style.backgroundColor = "#eef1f0";`
- Use `textContent` to change button words; do not recreate the button with `innerHTML`.
- A descriptive image alt: `alt="Book cover for The Lantern Orchard by Mere Kingsley"`
- If JavaScript cannot find an element: check spelling, `#`, `.`, and ensure the script is before `</body>` or uses `defer`.
- If nothing displays: open browser DevTools → Console; check URL, property names, missing brackets, quotes/backticks, and errors.
- If fetch fails: use `try/catch` and check `response.ok`.

10. Exam checklist

- HTML: title, stylesheet link, header/h1, main, footer, JS script link.
- JS: select element → create template literal → put it in the page.
- API: `async + await fetch() + await response.json()`.
- Cards: loop with `forEach`; use `html +=`; set `innerHTML` once after loop.
- CSS: flex for featured item; grid with `repeat(3, 1fr)` for cards.
- Interaction: event listener; add/remove/toggle a CSS class; update button text.
- Form/checkbox: label for matches input id; read `input.value` or `checkbox.checked`.
- Git: commit after each task with a descriptive message.